

KNOWLEDGE BASE

Joglo POS Backend

Panduan memahami sistem di balik aplikasi kasir restoran — dijelaskan untuk semua orang, tanpa perlu latar belakang teknis.

Versi 1.0 · Juni 2026

Aplikasi: Point of Sale (POS) Restoran · Joglo-Application

Daftar Isi

1. Apa Itu "Backend"? — Analogi Sederhana
2. Gambaran Besar: Bagaimana Semuanya Terhubung
3. Teknologi yang Digunakan (dan Kenapa)
4. Arsitektur Berlapis — Perjalanan Sebuah Permintaan
5. Struktur Modular — Setiap Fitur Punya "Kamar" Sendiri
6. Keamanan: Kartu Akses Digital (JWT) & Peran Pengguna
7. Alur Kerja Utama (dengan Contoh Nyata)
8. Daftar Fitur & Layanan (Endpoint)
9. Di Mana Sistem Ini "Tinggal"? (Deployment)
10. Glosarium — Kamus Istilah

Cara membaca dokumen ini: Anda tidak perlu bisa memrogram. Setiap konsep teknis disertai **analogi dunia nyata** (ditandai kotak hijau). Bagian berkode `seperti ini` hanyalah contoh — boleh dilewati jika tidak diperlukan.

1. Apa Itu "Backend"?

Aplikasi kasir yang dilihat dan disentuh kasir di layar tablet disebut **frontend** (bagian depan). Tetapi di balik layar, ada "otak" yang menyimpan data, menghitung, dan mengingat segala sesuatu. Itulah **backend**.

Analogi restoran: Bayangkan sebuah restoran.

- **Ruang makan** (yang dilihat pelanggan) = **Frontend** / aplikasi di tablet kasir.
- **Dapur + gudang + pembukuan** (tidak terlihat pelanggan) = **Backend**.

Pelanggan cukup memesan dari meja; mereka tidak perlu tahu cara dapur memasak atau bagaimana stok dicatat. Dapur menerima pesanan, memprosesnya, mengurangi bahan dari gudang, lalu mengirim hasilnya kembali.

Backend Joglo POS bertugas:

- Menyimpan semua data: menu, harga, stok bahan, pesanan, pembayaran, pengguna.
- Menghitung otomatis: total belanja, kembalian, dan **pengurangan stok bahan saat ada penjualan**.
- Menjaga keamanan: memastikan hanya orang berwenang yang bisa mengakses data tertentu.
- Melayani banyak perangkat sekaligus lewat internet.

Intinya: Frontend = wajah yang ramah. Backend = mesin yang bekerja keras di belakang. Keduanya berkomunikasi lewat internet, saling mengirim pesan singkat berisi data.

2. Gambaran Besar: Bagaimana Semuanya Terhubung

Ketika kasir menekan tombol di tablet, terjadi percakapan singkat lewat internet antara aplikasi dan backend. Berikut alur paling dasarnya:



Jawaban kemudian kembali ke arah sebaliknya: dari database → backend → internet → tablet, dalam hitungan milidetik.

Analogi memesan makanan lewat pelayan:

- Anda (tablet) memberi pesanan kepada **pelayan** (internet).
- Pelayan membawa pesanan ke **dapur** (backend).
- Dapur mengambil bahan dari **gudang** (database), memasak, lalu menyerahkan hasilnya kembali ke pelayan.
- Pelayan mengantar makanan kembali ke meja Anda.

Bahasa percakapan: "REST API"

Backend dan frontend berbicara dengan aturan baku yang disebut **REST API**. Sederhananya, ini adalah daftar "alamat layanan" yang bisa dipanggil, misalnya:

- **GET /menus** — "Tolong berikan daftar menu." (GET = mengambil data)
- **POST /pesanan** — "Tolong buatkan pesanan baru ini." (POST = mengirim/membuat data)

Setiap jawaban dikemas rapi dalam format yang konsisten, sehingga aplikasi selalu tahu apakah berhasil atau gagal:

Jika berhasil	Jika gagal
<pre>{ "success": true, "data": { ... } }</pre>	<pre>{ "success": false, "error": { ... } }</pre>

3. Teknologi yang Digunakan (dan Kenapa)

Setiap "bahan bangunan" dipilih dengan alasan. Berikut daftarnya dalam bahasa sederhana:

Teknologi	Perannya	Analogi
TypeScript	Bahasa pemrograman utama. Versi JavaScript yang lebih aman karena memeriksa kesalahan lebih awal.	Menulis dengan pengecekan ejaan otomatis menyala.
Hono.js	Kerangka kerja yang menerima & mengarahkan permintaan dari internet. Ringan dan cepat.	Resepsionis yang sigap mengarahkan tamu ke ruangan yang tepat.
PostgreSQL	Database — tempat semua data disimpan secara permanen dan rapi dalam bentuk tabel.	Lemari arsip raksasa yang sangat terorganisir.
Drizzle ORM	Penerjemah antara kode program dan database, agar data mudah diambil/disimpan tanpa salah.	Penerjemah bahasa yang fasih dua arah.
Zod	Pemeriksa data yang masuk. Menolak data yang salah bentuk sebelum diproses.	Satpam yang memeriksa kelengkapan dokumen di pintu masuk.
JWT	"Kartu akses digital" untuk membuktikan identitas pengguna yang sudah login.	Gelang masuk acara yang menandakan Anda sudah terdaftar.
Swagger	Dokumentasi interaktif — daftar semua layanan yang bisa dicoba langsung dari browser.	Buku menu lengkap dengan tombol "coba pesan".

Catatan: Pilihan teknologi ini mengutamakan **kecepatan, keamanan, dan kemudahan dikembangkan** di masa depan, sehingga fitur baru bisa ditambahkan tanpa membongkar yang sudah ada.

4. Arsitektur Berlapis — Perjalanan Sebuah Permintaan

Setiap permintaan yang masuk tidak langsung menyentuh data. Ia melewati beberapa **lapisan pemeriksaan** secara berurutan, seperti tamu restoran yang dilayani bertahap. Ini membuat sistem aman dan teratur.

1 Pintu Masuk (Routes) — Menentukan permintaan ini ditujukan ke layanan mana. Analogi: papan petunjuk arah.

2 Penjaga Keamanan (Middleware) — Memeriksa kartu akses (login) & hak (peran), serta mencatat aktivitas. Analogi: satpam + buku tamu.

3 Pemeriksa Formulir (Validation) — Memastikan data yang dikirim lengkap & benar. Analogi: petugas yang menolak formulir tak lengkap.

4 Penerima Tamu (Handler) — Menerjemahkan permintaan & menyiapkan jawaban. Analogi: pelayan yang mencatat pesanan.

5 Dapur (Service) — **Otak** tempat semua logika & perhitungan terjadi (hitung total, potong stok, dll). Analogi: koki yang memasak.

6 Gudang (Database) — Membaca & menyimpan data secara permanen. Analogi: gudang penyimpanan.

Mengapa berlapis? Jika ada masalah, kita tahu persis di lapisan mana mencarinya. Jika ingin mengubah cara perhitungan, cukup sentuh lapisan "Dapur" tanpa mengganggu yang lain. Ini membuat sistem **mudah dirawat** dan **tahan lama**.

Contoh nyata: satu permintaan melewati semua lapisan

Saat kasir membuat pesanan baru (`POST /pesanan`):

- Routes:** "Oh, ini menuju layanan Pesanan."
- Middleware:** "Kartu akses valid? Perannya kasir/admin? Boleh lanjut."
- Validation:** "Daftar item lengkap dan jumlahnya masuk akal? Oke."
- Handler:** "Saya terima, teruskan ke Dapur."
- Service:** "Hitung total, cek stok cukup, kurangi stok bahan, simpan pesanan." ← keajaiban terjadi di sini

6. **Database:** "Tersimpan permanen. Selesai."

5. Struktur Modular — Setiap Fitur Punya "Kamar" Sendiri

Backend tidak ditulis sebagai satu tumpukan besar yang berantakan. Ia dibagi menjadi **modul-modul terpisah**, satu untuk tiap area fitur. Masing-masing modul berisi 4 berkas dengan tugas jelas.

Berkas dalam modul	Tugasnya
schema	Mendefinisikan bentuk data yang valid (aturan pengisian).
routes	Mendaftarkan alamat layanan & siapa yang boleh mengaksesnya.
handler	Menerima permintaan & menyusun jawaban.
service	Logika inti & perhitungan.

Analogi gedung perkantoran: Tiap departemen (Keuangan, Gudang, HRD) punya ruangnya sendiri dengan staf & aturan masing-masing. Kalau perlu menambah departemen baru, tinggal buka ruangan baru — tidak perlu merombak seluruh gedung.

Modul yang sudah dibangun

Modul	Fungsi
Auth	Login & identitas pengguna.
Users	Mengelola akun pengguna (admin).
Menus (+ Resep)	Daftar menu jual & komposisi bahan tiap menu.
Bahan Baku	Master data bahan & stoknya.
Suppliers	Data pemasok bahan.
Pesanan	Transaksi penjualan (otomatis memotong stok).
Pembayaran	Pembayaran pesanan & kembalian.
Pesanan Bahan	Pemesanan pembelian ke pemasok (Purchase Order).
Transaksi Masuk	Penerimaan bahan (stok bertambah).
Transaksi Keluar	Pengeluaran bahan (penjualan otomatis + manual: rusak/kedaluwarsa).

6. Keamanan: Kartu Akses Digital & Peran Pengguna

Login & "Kartu Akses" (JWT)

Saat pengguna login dengan benar, backend memberikan sebuah **token** — semacam kartu akses digital sementara. Untuk setiap permintaan berikutnya, aplikasi menunjukkan kartu ini sebagai bukti "saya sudah login dan berhak".

Analogi gelang masuk konser: Anda menunjukkan tiket sekali di pintu (login), lalu diberi gelang (token). Setelah itu, cukup tunjukkan gelang untuk masuk-keluar area — tak perlu beli tiket berulang. Gelang punya masa berlaku & tak bisa dipalsukan.

Kata sandi pengguna **tidak pernah disimpan apa adanya**. Ia diacak (di-hash) sehingga meski database bocor, kata sandi asli tetap tak terbaca.

Peran Pengguna (Role)

Tidak semua orang boleh melakukan segalanya. Ada 3 peran:

Peran	Boleh apa
Admin	Akses penuh: kelola menu, bahan, pengguna, inventaris, transaksi.
Kasir	Operasional penjualan: buat pesanan, terima pembayaran, lihat menu.
Owner	Pemilik: melihat data & laporan (peran pemantauan).

Contoh aturan: Seorang **kasir** bisa membuat pesanan, tetapi **tidak bisa** mengubah daftar menu atau menambah pengguna baru — itu hanya hak **admin**. Backend menolak otomatis jika peran tak sesuai.

7. Alur Kerja Utama (dengan Contoh Nyata)

Berikut beberapa alur terpenting, diceritakan langkah demi langkah.

Alur A — Login

KIRIM

POST /auth/login

1. Kasir memasukkan username & kata sandi.
2. Backend memeriksa kecocokan dengan data tersimpan (kata sandi yang teracak).
3. Jika benar → backend mengirim **kartu akses (token)**. Jika salah → ditolak dengan pesan jelas.
4. Aplikasi menyimpan token untuk dipakai pada semua aksi berikutnya.

Alur B — Menampilkan Menu

AMBIL

GET /menus

perlu login

1. Aplikasi meminta daftar menu (bisa disaring & dibagi per halaman).
2. Backend mengambilnya dari database & mengembalikan nama, harga, kategori, status aktif.
3. Tablet menampilkannya sebagai grid produk untuk dipilih kasir.

Alur C — Membuat Pesanan 📄 (fitur inti)

KIRIM

POST /pesanan

login

admin/kasir

Ini bagian terpenting. Semua terjadi dalam satu tindakan **menyeluruh atau batal total** (tidak setengah-setengah):

1. Kasir memilih item + jumlah, lalu menyimpan pesanan.
2. Backend **menghitung total sendiri** dari harga resmi (tidak mempercayai harga dari aplikasi — demi keamanan).
3. Backend melihat **resep** tiap menu, lalu menghitung total bahan yang dipakai.
4. Mengecek stok cukup. Jika kurang → pesanan **ditolak** dengan pesan jelas.
5. Jika cukup → **stok bahan otomatis berkurang**, pesanan tersimpan, & pemakaian bahan dicatat.

Analogi: Memesan 2 kopi otomatis mengurangi stok biji kopi & susu di gudang sesuai resep — tanpa kasir menghitung manual. Jika stok susu habis, sistem menolak

sebelum transaksi terjadi.

Alur D — Pembayaran

KIRIM

POST /pembayaran

1. Kasir memilih metode (tunai/QRIS/debit) & memasukkan jumlah bayar.
2. Backend memastikan jumlah bayar $\geq$ total, lalu **menghitung kembalian**.
3. Status pesanan berubah menjadi **"selesai" (completed)**.
4. Tidak bisa dibayar dua kali — sistem menolak pembayaran ganda.

Alur E — Membeli Bahan ke Pemasok (Inventaris)

KIRIM

POST /pesanan-bahan → .../receive

1. Admin membuat pesanan pembelian (PO) ke pemasok — statusnya "menunggu".
2. Saat barang datang, admin menekan **"terima"**.
3. Backend otomatis **menambah stok** bahan & mencatat harga beli terbaru.
4. Jika ada bahan rusak/kedaluwarsa, dicatat lewat **Transaksi Keluar** (stok berkurang dengan alasan).

Lengkap dua arah: stok **berkurang** otomatis saat menjual, & **bertambah** saat menerima pembelian. Semua pergerakan tercatat rapi.

8. Daftar Fitur & Layanan (Endpoint)

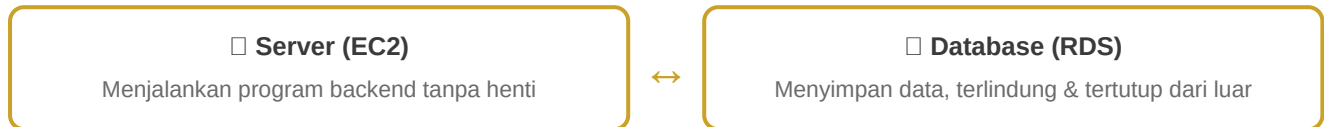
Berikut ringkasan layanan yang tersedia. **AMBIL** = membaca data, **KIRIM** = membuat/mengubah data.

Area	Layanan	Keterangan
Auth	KIRIM /auth/login AMBIL /auth/me	Login & cek identitas.
Users	AMBIL KIRIM /users	Kelola akun (admin).
Menu	AMBIL KIRIM /menus /menus/:id/resep	Menu jual & resepnya.
Bahan Baku	AMBIL KIRIM /bahan-baku	Stok bahan.
Supplier	AMBIL KIRIM /suppliers	Data pemasok.
Pesanan	KIRIM /pesanan /pesanan/:id/cancel	Penjualan + auto potong stok.
Pembayaran	KIRIM /pembayaran	Bayar + kembalian.
Purchase Order	KIRIM /pesanan-bahan /.../receive · /.../cancel	Beli bahan ke pemasok.
Transaksi Masuk	KIRIM /transaksi-masuk	Terima bahan → stok naik.
Transaksi Keluar	KIRIM /transaksi-keluar	Bahan rusak/adjustment → stok turun.

Dokumentasi hidup: Semua layanan ini dapat dilihat & dicoba langsung lewat halaman **Swagger** di alamat **/docs** pada server — seperti "buku menu interaktif" untuk pengembang.

9. Di Mana Sistem Ini "Tinggal"?

Backend tidak berjalan di tablet kasir, melainkan di sebuah **komputer server** yang menyala 24 jam di pusat data (cloud) milik Amazon (AWS), sehingga bisa diakses dari mana saja lewat internet.



Analogi: Server (EC2) ibarat **kantor pusat** yang selalu buka. Database (RDS) ibarat **brankas di ruang dalam** — hanya bisa diakses dari kantor pusat, tidak langsung dari jalanan, demi keamanan data.

Bagaimana pembaruan diterapkan?

Saat ada perbaikan atau fitur baru, kode dikirim ke pusat penyimpanan kode (GitHub), lalu server "menarik" versi terbaru & dijalankan ulang. Dokumentasi (Swagger) ikut diperbarui otomatis.

Aman: data sensitif (kata sandi database, kunci rahasia) disimpan terpisah di server & tidak pernah ikut tersimpan di kode publik.

10. Glosarium — Kamus Istilah

Istilah	Arti sederhana
Backend	"Otak" sistem di balik layar yang menyimpan data & menghitung.
Frontend	Aplikasi yang dilihat & disentuh pengguna (tablet kasir).
API	Daftar "layanan" yang bisa dipanggil aplikasi untuk meminta sesuatu ke backend.
Endpoint	Satu alamat layanan tertentu, mis. <code>/menus</code> .
Database	Tempat penyimpanan data permanen, tersusun dalam tabel.
Server	Komputer yang menyala terus untuk menjalankan backend.
Token / JWT	"Kartu akses digital" pembukti identitas setelah login.
Role / Peran	Tingkat hak akses pengguna (admin, kasir, owner).
Resep	Daftar bahan + jumlah yang dipakai untuk membuat satu menu.
Stok	Jumlah bahan yang tersedia di gudang.
Transaksi atomik	Serangkaian langkah yang berhasil semua atau gagal total — tak ada kondisi setengah jadi.
Deployment	Proses menempatkan & menjalankan program di server agar bisa dipakai.
Swagger	Halaman dokumentasi interaktif untuk melihat & mencoba semua layanan API.
Cloud (AWS)	Pusat data sewaan tempat server & database berjalan.

Penutup: Backend Joglo POS dirancang agar **aman, teratur, dan mudah dikembangkan**. Dengan memahami analogi "restoran + dapur + gudang" di dokumen ini, siapa pun dapat membayangkan bagaimana setiap penjualan, pembayaran, dan pergerakan stok diproses di balik layar — tanpa perlu membaca satu baris kode pun.

— Akhir Dokumen —

Joglo POS Backend · Knowledge Base v1.0 · Juni 2026